

LC 3208: Alternating Groups II

Google

Samsara

Sliding Window

Circular Array

Problem Concept

- **Goal:** Return the number of valid alternating groups of size k in a circular array.
- **Sliding Window Approach:** To process a sliding window on a circular array efficiently, we iterate the window up to $n + k - 1$.
- **Modulo Arithmetic:** We map overflowing indices back to the valid array range using modulo arithmetic. For example, instead of accessing index $j - k$ directly, we use $(j - k) \% n$.
- **Error Tracking:** We maintain a *bad pair count* (where consecutive elements are equal). Whenever consecutive elements are equal, we increment the error count. A group is valid and alternating only when $error == 0$, at which point we increment our answer.

Example Breakdown

Array: 0 1 0 0 1 0 1 | $k = 6$

Notice the consecutive identical elements forming a "bad pair" at indices 2 and 3 (0 0).

The valid alternating groups formed by wrapping around the circular array are:

- 0 1 0 1 0 1 (wrapping forward from the end)
- 1 0 1 0 1 0

C++ Solution

```
class Solution {
public:
    int numberOfAlternatingGroups(vector<int>& colors, int k) {
        int n=colors.size();

        int err=0;
        int ans=0;
        for(int i=0;i<k;i++){
            if(i>0 && colors[i]==colors[i-1])
            {
                err++;
            }
        }
        if(err==0) ans++;

        for(int j=k;j<n+k-1;j++){
            if(colors[(j-k)%n]==colors[(j-k+1)%n]){
                err--;
            }

            if(colors[j%n]==colors[(j-1)%n]){
                err++;
            }

            if(err==0) ans++;
        }
        return ans;
    }
};
```